

MSME Lending Decision System – Architecture Decisions & Trade-offs

Project Overview

This project implements a full-stack MSME lending decision platform built using React, Node.js, PostgreSQL, and MongoDB. The objective was to create a lightweight but production-oriented system capable of collecting business information, processing loan applications, and generating transparent credit decisions based on a custom rule-based scoring engine.

Architecture

The application follows a layered architecture consisting of presentation, API, business logic, and persistence layers. The React frontend communicates with an Express REST API. PostgreSQL stores structured entities such as business profiles and loan applications, while MongoDB stores flexible decision documents and audit logs. This separation allows each database to be used for the type of data it handles best.

Design Decisions

Express was selected because it is lightweight and modular. Sequelize simplified relational database operations and relationships, while Mongoose provided an efficient document model for decision history. Joi validation was implemented on the backend to ensure consistent API validation, and React performs client-side validation for a better user experience. The decision engine was implemented as a pure service so that it remains reusable, testable, and independent of controllers or databases.

Credit Decision Logic

The scoring engine evaluates five independent signals: revenue-to-EMI ratio, loan-to-revenue multiple, tenure risk, business type, and fraud checks. Each signal contributes a weighted score that is converted into a credit score ranging from 300 to 850. The system also returns human-readable reason codes so that users understand why a loan was approved or rejected. This improves transparency compared to a black-box decision model.

Trade-offs

To complete the assignment within one day, several practical trade-offs were made. EMI was calculated using simple division rather than the standard interest-based EMI formula, making the logic easier to understand. The scoring model is rule-based instead of machine-learning driven, ensuring explainability and easier debugging. Database synchronization through Sequelize was used instead of migrations to accelerate development.

Deployment

The frontend was deployed on Render as a static site, while the backend was deployed as a Node.js web service. PostgreSQL was migrated from a local instance to Neon, and MongoDB was migrated to MongoDB Atlas to support cloud deployment. Environment variables were managed securely through Render instead of local configuration files.

Challenges

The primary challenges involved deployment. Local database configurations caused connection failures in the cloud environment, requiring migration to hosted databases. MongoDB authentication, PostgreSQL connectivity, CORS configuration, and environment variables were resolved during deployment. These issues provided valuable experience with production deployment and cloud infrastructure.

Future Improvements

With additional time, I would implement JWT authentication, asynchronous decision processing using queues, comprehensive automated testing, real EMI calculations with interest rates, integration with

external credit bureau APIs, richer dashboards for application history, and CI/CD pipelines for automated deployments.

Conclusion

The project demonstrates full-stack development skills, REST API design, database modeling, validation, deployment, and system design. While simplified in some areas to meet the assessment timeline, the application follows clean architecture principles and provides a strong foundation for a production-ready MSME lending platform.